

UNIVERSITÀ POLITECNICA DELLE MARCHE
FACULTY OF ENGINEERING

Department of Information Engineering

Master in Computer and Automation Engineering



PREVENTIVE MAINTENANCE FOR ROBOTICS AND INTELLIGENT AUTOMATION
EXAMINATION PROJECT

☞ Addressing PHM North America 2024 Challenge with
Kolmogorov-Arnold Networks

Authors

Valerio Morelli
Federica Paganica
Federico Staffolani
Enrico Maria Sardellini

ACADEMIC YEAR 2024-2025

1 PHM North America 2024 Challenge	1
1.1 The goal of the challenge	1
1.2 The structure of the data set	1
2 How to run the project	3
2.1 Environment set up	3
2.1.1 Prerequisites	3
2.1.2 Install the requirements and run the Jupyter server	3
3 Structure of the repository and results obtained	5
3.1 Explorative data analysis and data preprocessing	6
3.1.1  Skewness study and PCA	6
3.1.2  Multicollinearity study with VIF	7
3.1.3  Feature extraction	7
3.1.4  Correlation pruning	8
3.1.5  Cherry picking	9
3.2 Training a KAN to predict a PDF of the $torque_{target}$	9
3.2.1  Torque target probabilistic regression	9
3.2.2  Using learned regressor to predict torque target	10
3.3 Training a KAN for fault detection	10
3.3.1  Fault detection	10
3.3.2  Generating faulty predictions	10
3.3.3  Explaining KAN with Shapley value	11
3.4 Qualitative evaluation	11
3.4.1  Data drift and outliers analysis	12
3.4.2  PCA analysis	12
3.4.3  t-SNE analysis	13
3.4.4  KNN in PCA domain	13
A The Kolmogorov-Arnold Theorem in Machine Learning	16
A.1 The Kolmogorov-Arnold representation theorem	16
A.2 The Kolmogorov-Arnold Networks	17
A.2.1 The paper's real contribution	17
A.2.2 The high interpretability of KAN	18
A.2.3 The introduction of MultKANs as a generalization of KANs	18

List of Figures

3.1	Overview of the workflow	5
3.2	Healty and faulty distributions	6
3.3	Clusters given by np/ng ratio	7
3.4	The VIF test highlights how some features can be linearly predicted by others with a high degree of R^2	7
3.5	Differences between the original correlation matrix and the pruned one	8
3.6	The result of Cherry-picking ranking for regression and classification.	9
3.7	Although more accurate, the MLP is much less interpretable, even for small layer sizes. For all intents and purposes, it can be considered a black box.	10
3.8	The function learned by the KAN for predicting the PDF of $torque_{target}$	11
3.9	A low variance of the resulting test loss is an empirical justification for the workflow.	11
3.10	The prediction of the mean of $torque_{target}$ is all in all quite accurate.	12
3.11	A KAN with a multiplication node, here referred to as "MultiKAN" and one without, referred to as "PyKAN" have been trained for fault detection. Whereas the former achieves a greater test score of 0.89, the latter is smaller and then more interpretable, but with a lower test score of 0.82.	13
3.12	The 10 trainings show a low variance for the test loss.	13
3.13	The prediction of the faulty state is a bit more disappointing than that of the $torque_{target}$	14
3.14	The Shapley value, computed for 2.000 random samples, shows how the KAN weights each feature in its predictions.	14
3.15	A 3D PCA plot shows the regions of the faulty samples of the test set alongside those of the train set.	15
3.16	The distribution of the KAN's predictions over the test set are plotted next to those of the geometric classifiers. This gives an idea of how much test samples predicted to be faulty dwell near the faulty samples of the train set.	15
A.1	A simple two-layer KAN, where the activation functions are arranged on the edges and the nodes compute the sum. The number of outer functions does not comply with the KAT statement, since they are less than $2n + 1 = 9$	17
A.2	After training with sparsification penalty, the nodes with <i>incoming</i> and <i>outgoing</i> scores — defined as the minimum and maximum $L1$ norm for their activation functions — below a certain threshold are removed.	18

PHM North America 2024 Challenge

This project proposes an innovative solution for the PHM North America 2024 Challenge based on the application of the brand new Kolmogorov-Arnold Neural Network, which is based on the homonymous representation theorem. In this first chapter, we'll have a look at what this challenge is all about.

1.1 The goal of the challenge

The *PHM North America 2024 Conference Data Challenge* deals with estimating the health of helicopter turbine engines. Challengers are requested first to solve probabilistic regression to predict the mismatch between the controller's torque demand and the engine's actual torque output, namely the $torque_{margin}$ ¹. This is then fed into a fault detection classifier along with the other features in the dataset.

Note

The challenge explicitly requires a confidence metric for both regressor and classifier predictions. Now, while a classifier naturally outputs a confidence score, a regressor model will most likely answer with a single number. Therefore, *probabilistic regression*^a is needed instead. The nature of the distribution whose parameters are to be predicted by the regressor is up to the challenger.

^aIn a way, the model needs to learn functions that map a sample to a PDF.

1.2 The structure of the data set

The data set provided collects samples from seven engines of the same make and model. Four of these are used to record the 742.625 samples of the training set and the remaining three are used for the test and validation sets, made of 21.436 samples each.

Unfortunately, no time dimension was given in the dataset, so there's no way of knowing whether groups of consecutive samples correspond to consecutive measures².

¹It is defined as

$$torque_{margin} := 100 \times \frac{torque_{measured} - torque_{target}}{torque_{target}}$$

²Actually, Ozeki and Nakamura [2024] introduce k-NN smoothing that rebuilds the missing time information using the distance in the feature space. This works under the assumption that the features change smoothly over time.

 Note

As is often the case in automation, we use the term "validation set" to refer to what is often called a "train set" in artificial intelligence, and vice versa. However, such a misnomer is not a problem in this context, as we will use the term "test set" to refer to the union of the challenge's validation and test sets.

Another problem is the lack of ground truth for both the validation and test sets. Therefore, we decided to merge them into one test set and keep it for qualitative analysis only. The train set is therefore split using the *holdout* method into three distinct labelled train, validation and test sets.

The features and labels are described in the 1.1 and 1.2 tables.

Table 1.1: The 7 features of the data set.

Field	Description
<code>trq_measured</code>	Actual engine torque output (%)
<code>oat</code>	Temperature of the external air (°C)
<code>mgt</code>	Average temperature inside the turbine (°C)
<code>pa</code>	Height above the standard atmospheric pressure level (feet)
<code>ias</code>	Speed of the helicopter relative to the surrounding air (knots)
<code>np</code>	Effective power output of the engine (%)
<code>ng</code>	Speed of the engine's compressor (%)

Table 1.2: The labels for the regression and classification tasks.

Field	Description
<code>faulty</code>	Engine faulty state (0 = <i>healthy</i> , 1 = <i>faulty</i>)
<code>trq_margin</code>	Indicates if the engine is running exactly as expected or not

How to run the project

This chapter explains how to use the repository to reproduce all the experiments described in Chapter 3.

The repository is available at [📄](#).

2.1 Environment set up

The repository is made of multiple Jupyter notebooks, divided into the 4 sections shown in Figure 3.1. To shrink these notebooks and make them as readable as possible, the core parts have been extracted inside the `src` module.

🔗 Important

Before running the notebooks, make sure to extract the `/dataset/0-original/dataset.zip` archive. Furthermore, as I haven't included the `.csv` files due to their size, the notebooks should be run in sequence one after the other.

2.1.1 Prerequisites

- Python 3.7+,
- Optionally, *virtualenv* for managing environments.

2.1.2 Install the requirements and run the Jupyter server

Clone the repository with `git clone https://github.com/...`, then, from the root of the repository follows the following steps.

1. Optionally, create and enter a virtual environment with `python3 -m venv venv` and then `source venv/bin/activate` or `venv/Scripts/activate` on Windows.
2. Run `pip install -r requirements.txt`.
3. Run `jupyter lab` in the root of the repository to enter the web-based jupyter environment or use the tools provided by your favourite IDE.

 Note

To give an idea of how much time each cell takes to complete, we have given the execution time, based on *i7-10750H* CPU, as a comment above each cell. **No GPU is required to run these notebooks**, even the training should work fine on the CPU, as the networks involved are very small.

You can now begin running the notebooks in sequence, starting from 1.1.

Structure of the repository and results obtained

In this chapter, the structure of the workflow is presented. We begin with preprocessing and then move on to the training of the networks for the regression and classification task. Finally, a qualitative analysis of the results obtained is provided.

Figure 3.1 shows the strategy devised to solve the challenge. In the first section, data preprocessing, some features are derived from the original 7 and then pruned and selected. These are used in both sections 2 and 3 to train the regression and classifier models. The output of the former is also used as input for the latter. Finally, in section 4, we analyze the generalizability of the model over the unlabeled test set.

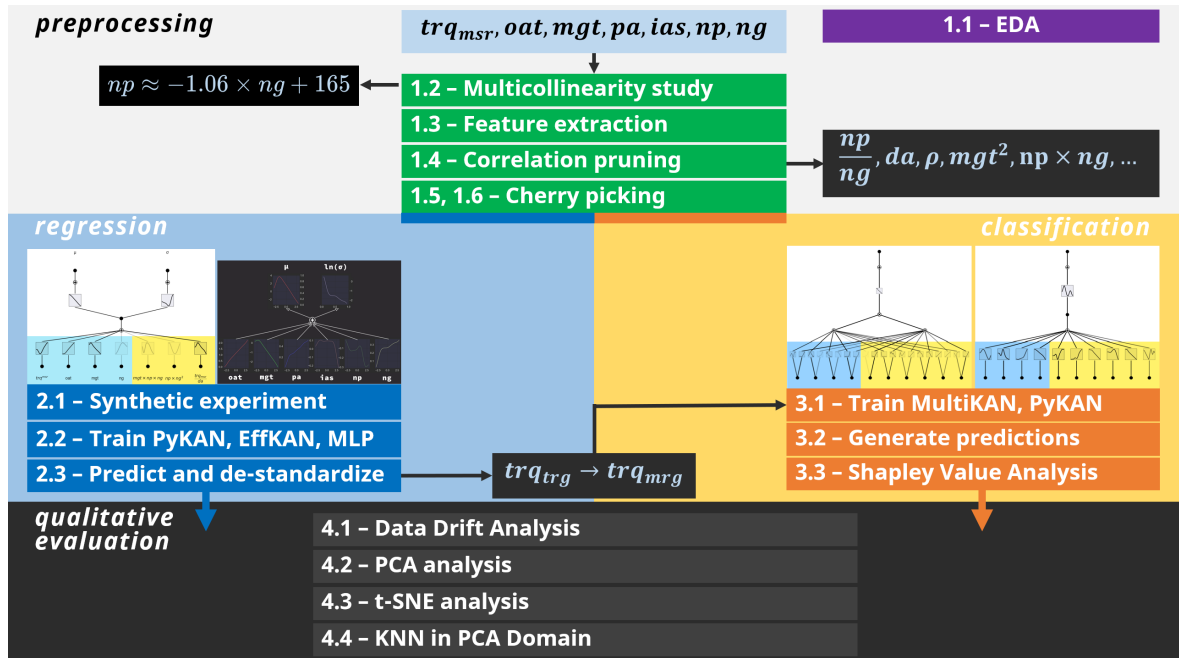


Figure 3.1: Overview of the workflow

3.1 Explorative data analysis and data preprocessing

3.1.1 Skewness study and PCA

The first part of the exploratory analysis focused on the determination of the *skewness* of each field in order to distinguish between healthy and faulty values.

From the observation of Figure 3.2, it can be deduced that *pa* tends toward higher values in faulty conditions. Conversely, *np* and *ng* show a trend toward slightly lower values. Thus, a decrease in these values is symptomatic of a fault condition. In addition, *trq_measured*, *mgt* and *oat* show significant overlap, which indicates a lower significance in fault detection.

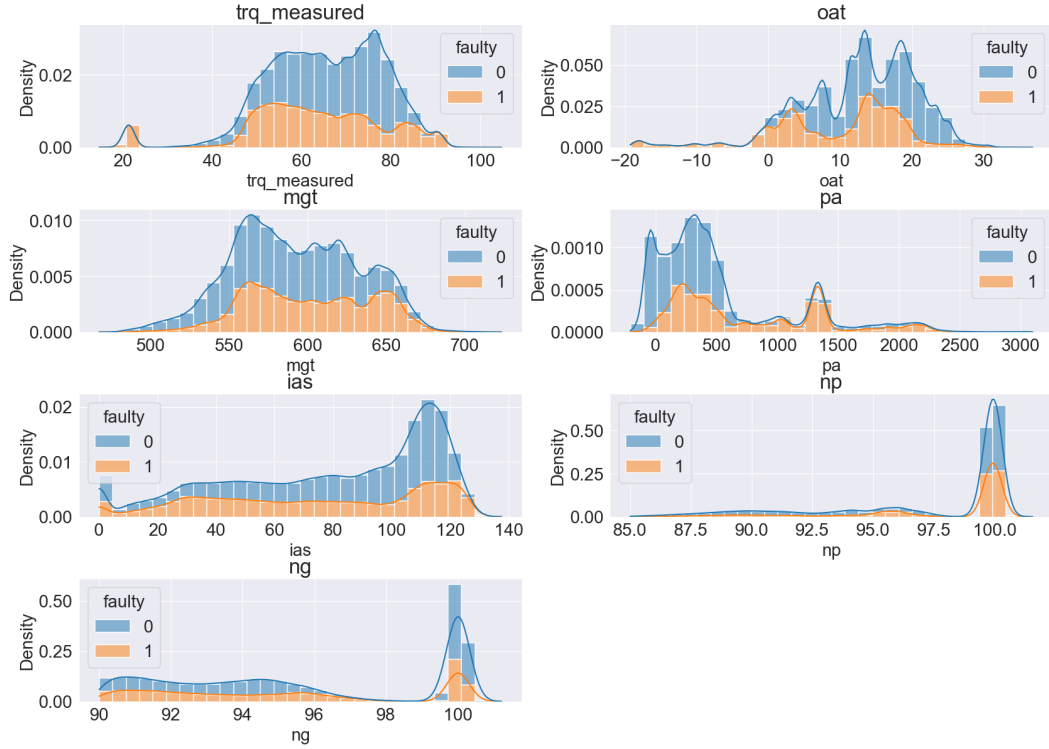


Figure 3.2: Healty and faulty distributions

Subsequent analysis focused on determining whether there was a strong correlation between features by examining the *correlation matrix*. In general, there is a very low correlation among them.

The only exception is *np* and *ng* which have a negative correlation of 0.79. Based on these results, it was decided to discard the idea of merging two or more features into one.

After that, the technique of *PCA* was applied. Specifically, we chose two and three components that cover about 80% of the variance.

Here we find out that the dataset is made of 2 *distinct clusters* of samples, one for samples having $np < ng$ and the other for samples having $np > ng$, as can be seen in Figure 3.3.

In the last analysis we went to consider whether possible *random sampling* would generate any criticality.

We also demonstrated that when comparing the skewness of the original dataset with a 10% random subsample, the trend of the curves in each feature almost always overlaps, allowing the possibility of random sampling to train the different models more quickly.

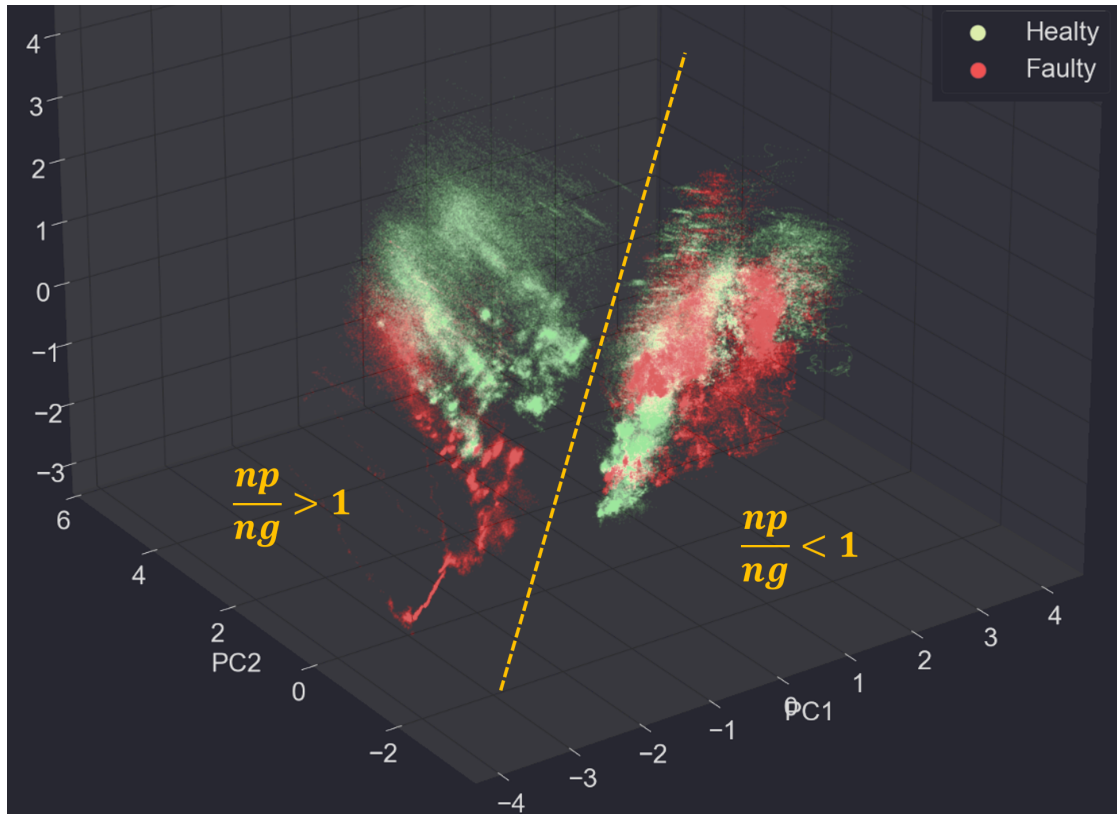


Figure 3.3: Clusters given by np/ng ratio

3.1.2 Multicollinearity study with VIF

Running the *Variance Inflation Factor* test, in figure 3.4, it becomes evident that **the original dataset is affected by a multicollinearity problem: np and ng have a strong linear dependency**¹. This is the source of a rank deficiency problem in the matrix of the data samples.

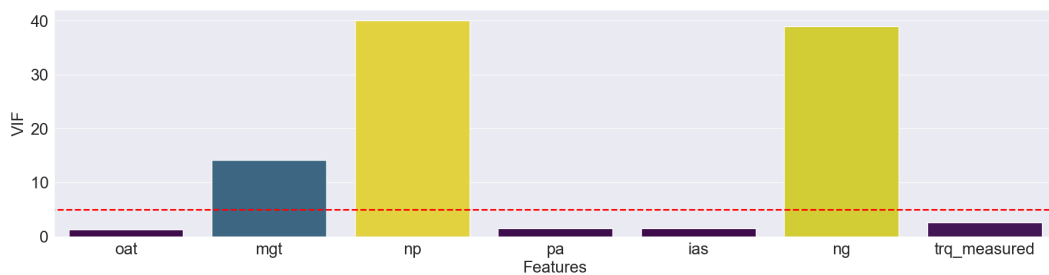


Figure 3.4: The VIF test highlights how some features can be linearly predicted by others with a high degree of R^2 .

The *negative* correlation is explained physically by the structure of the engine: there is a power balance between the compressor-turbine system and the power turbine. When the rotor draws a lot of energy — as if adding load to an electric generator — less is left for the compressor and vice-versa.

3.1.3 Feature extraction

From the original 7 features, we have derived the following new features.

¹If we choose np as the dependent variable, we get the approximation $np \approx -1.06 \times ng + 165$.

- The first added feature was the np/ng ratio.
- Then, we added all the polynomial combinations of features up to the 3rd degree.
- Next, *air density* was calculated by two different methods. The air density formula (da) was calculated as a combination of pressure altitude and outside air temperature converted to Kelvin, as

$$da = 1.2376 \times pa + 118.8 \times (oat + 273.15) - 1782$$

- The density of air ρ was then derived from the perfect gas law as

$$\rho = \frac{P}{R_s \cdot (oat + 273.15)}$$

where

$$P = P_0 \times \left(1 - \frac{a \times (pa \times 0.3048)}{T_0} \right)^{\frac{g}{R \times a}}$$

and T_0 , P_0 , a , g , R and R_s are well known constants.

- Finally, two additional columns are created for each feature by *normalizing* the original 7 features first with da and then with ρ . This was done to reduce the dependency on environmental conditions.

We end up with a total of 81 features, a far too large amount to handle, so a feature selection is needed.

3.1.4 🔄 Correlation pruning

Prior to feature selection, we also decided to prune highly correlated features, as they are most likely to be redundant, as shown in Figure A.2.

Other than the non-highly correlated extracted features, we also retained all the original features except for np , which was shown to be linearly dependent on ng .

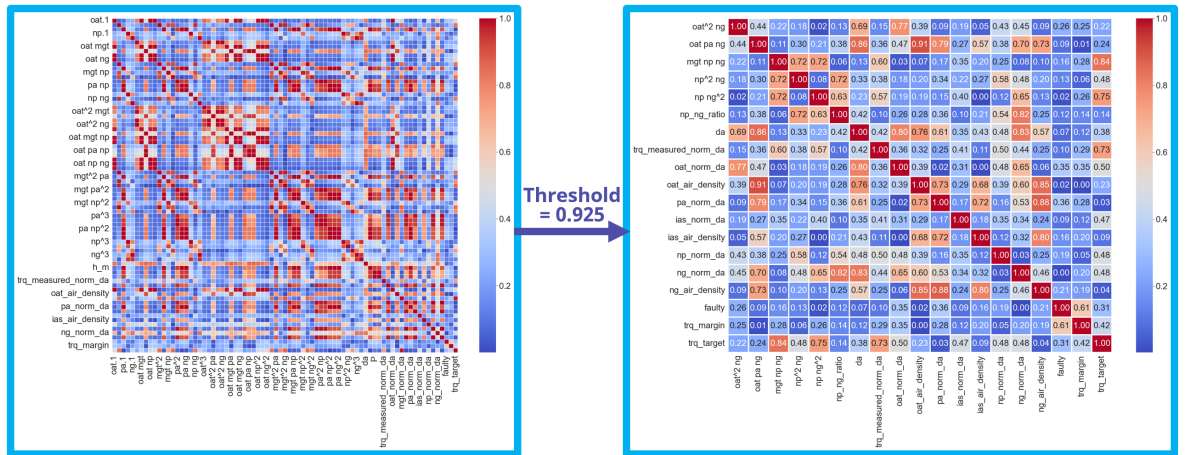


Figure 3.5: Differences between the original correlation matrix and the pruned one

It is interesting to note that, in the original correlation matrix, there is an almost perfect correlation between features normalized to *air_density*. This occurs because they have a small variance compared to that of ρ .

3.1.5 Cherry picking

In line with the goal of interpretability, we chose cherry-picking over embedding, otherwise the selected feature would not be as physically interpretable. The decision boundary for culling was chosen using not one, but three synthetic metrics: Pearson correlation with ground truth, *ANOVA* and *Kruskal-Wallis* tests.

Figure 3.6 shows the overall ranking result, computed as the average of the single rankings. We selected the *top 3* features for regression and the *top 5* features for classification.

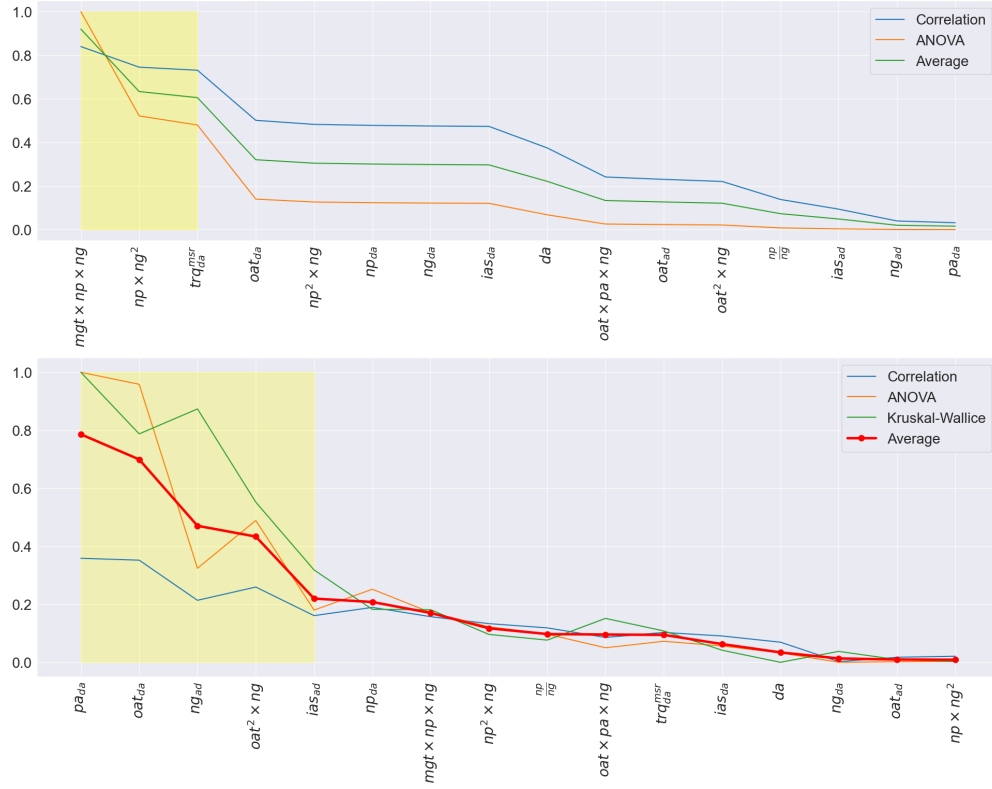


Figure 3.6: The result of Cherry-picking ranking for regression and classification.

3.2 Training a KAN to predict a PDF of the $\text{torque}_{\text{target}}$

3.2.1 Torque target probabilistic regression

Three different models have been considered: *PyKAN* [Liu, 2024] and *EfficientKAN* [Blealtan, 2024], which are two different implementations of a Kolmogorov-Arnold Network and *MLP*.

The goal is to prove the superiority of KAN networks over MLP in terms of interpretability.

They were trained with a learning rate of 0.05 and for 30 epochs, except *MLP* which was trained for 100 epochs.

The *MLP*, shown in figure 3.7, trained like Romano [2024], achieve a test *GaussianNLLLoss* loss of -4.65 , far lower than the -3.34 loss which the KANs converge to. Figure 3.8 shows the configuration that the *PyKAN* learns.

As discussed earlier, only the training set is labelled. Therefore, it was necessary to use the holdout method to select the validation and test sets. Therefore, the training was repeated 10 times to assert the dependency on the random choice of the training set. The result is shown in Figure 3.9.

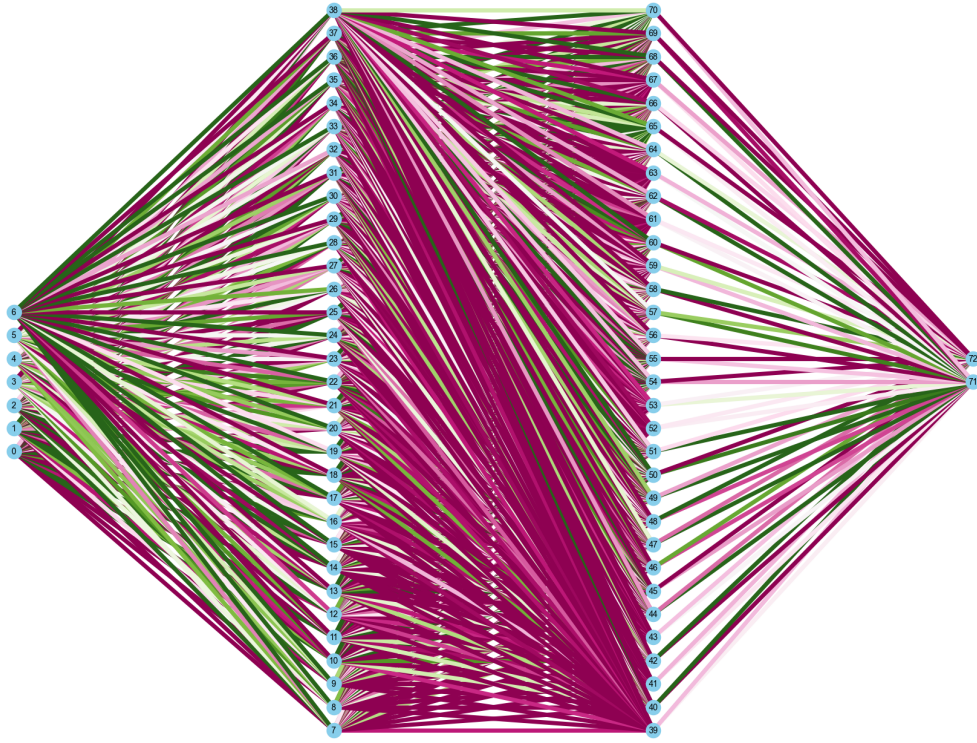


Figure 3.7: Although more accurate, the MLP is much less interpretable, even for small layer sizes. For all intents and purposes, it can be considered a black box.

3.2.2 🔄 Using learned regressor to predict torque target

The best KAN, out of the 10 trained, was loaded, along with the means and standard deviations of the features of its randomly selected train set. We then applied the same standardization to the entire dataset, de-standardized the predicted $torque_{target}$, shown in Figure 3.10, and converted it to $torque_{margin}$, which was used as input to the classifier.

3.3 Training a KAN for fault detection

3.3.1 🔄 Fault detection

Having already settled on the PyKAN architecture, we trained it much like the one in regression. We considered two configurations, one with a multiplication node and one without, shown in Figure 3.11. The former achieves the greater test score² of 0.89.

What we discussed in regression about the 10 trainings, still holds here, as proved by figure 3.12.

3.3.2 🔄 Generating faulty predictions

As before, the checkpoint of the best MultiKAN was loaded and the prediction was generated over the entire dataset, using the same standardization as during training.

²The score is defined by the challenge website, giving more penalty for false negatives phmsociety [2024].

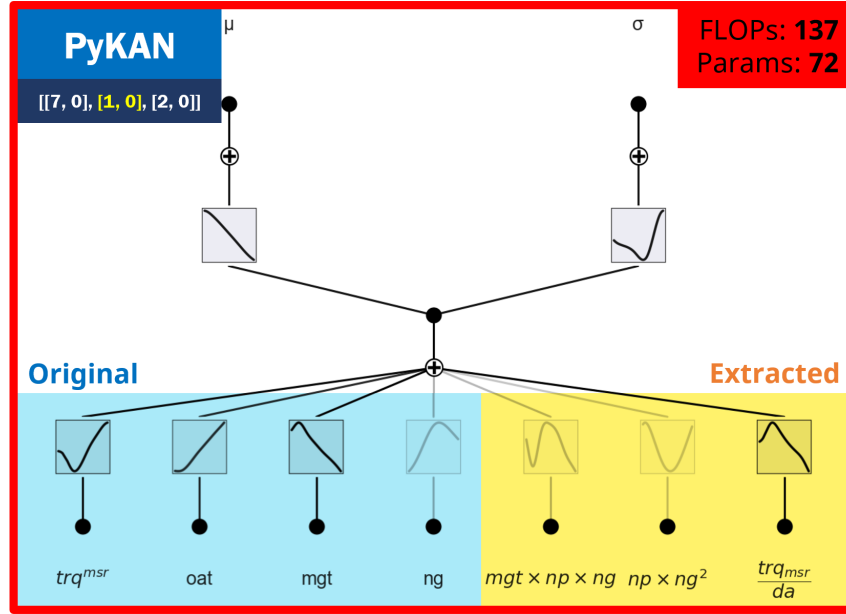


Figure 3.8: The function learned by the KAN for predicting the PDF of $torque_{target}$.

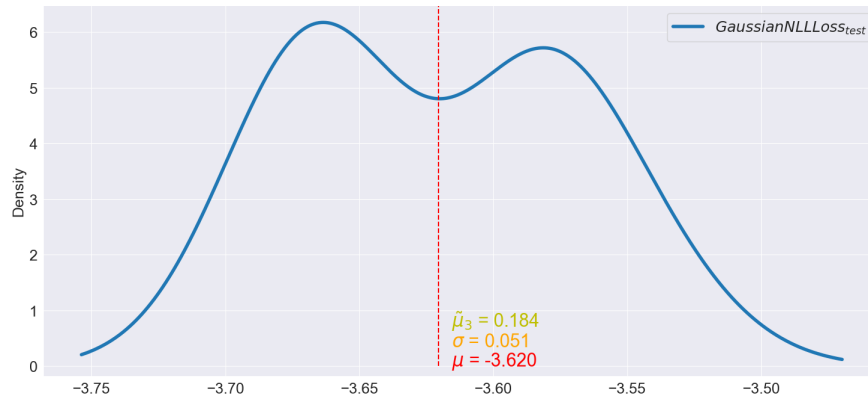


Figure 3.9: A low variance of the resulting test loss is an empirical justification for the workflow.

3.3.3 Explaining KAN with Shapley value

The Shapley value is a powerful tool in the field of eXplainableAI. Through an iterative approximation, the features of the samples are randomly permuted and the distance in their prediction is used to understand how the model weights each of them to assert the faulty state of the engine. As shown in Figure 3.14, **the network has learned to give highest priority to the $torque_{margin}$ predicted by the regression KAN.** This is very much in line with what one would think, since the $torque_{margin}$ gives a measure of how much the engine is matching the controller's request.

3.4 Qualitative evaluation

The next sections are devoted to qualitative analysis, with the goal of assessing if PyKAN actually generalize over the test set. As said, we don't have the ground truth for this yet, so a qualitative analysis is needed.

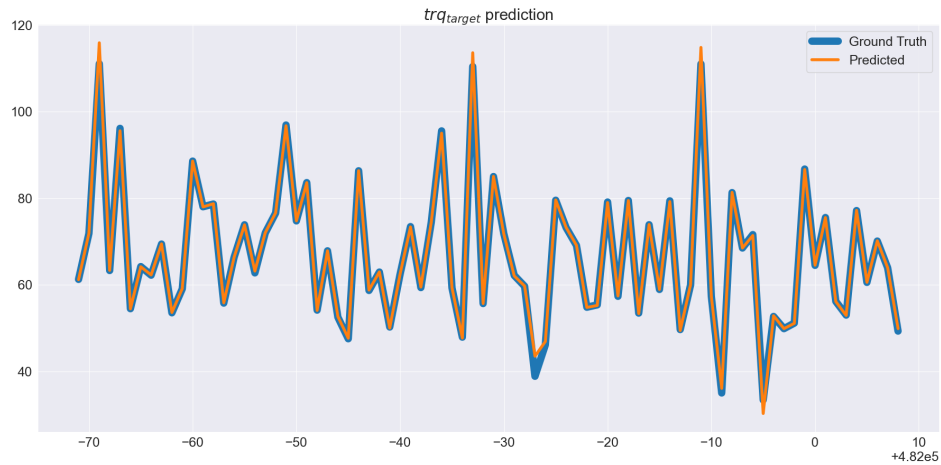


Figure 3.10: The prediction of the mean of $\text{torque}_{\text{target}}$ is all in all quite accurate.

3.4.1 Data drift and outliers analysis

Data drift has been shown to have a significant impact on the test set. The distribution of the KAN prediction across unseen feature regions is a subject of investigation.

We start this study by showing the stacked distributions of the healthy and faulty predictions. From that, it can be concluded that the claim that high values of trq_margin are a sign of a faulty engine is contradicted by the data, which shows that high values do not uniquely correspond to faulty situations.

Finally, an analysis of outliers in the trq_margin variable is provided, using the *Interquartile Range Method*. In total, this method detected 15.746 outliers for the training set (all with negative values, related to faulty engines) and only 1 for the test set, with an extremely high value.

In addition, we observe that some features are also above the 95% percentile, indicating that this engine is under abnormal condition. The table below shows these ranges.

Table 3.1: Outlier Values and Ranges (Test Set)

Feature	Outlier Value	Range (5-95%) on Test Set
oat	19.5	0.0 - 25.75
trq_measured	96.0	50.7 - 87.3
mgt	671.0	542.955 - 656.9
pa	187.7568	144.78 - 855.406
ias	27.25	3.847 - 125.625
np	100.24	99.69 - 100.22
ng	98.65	90.18 - 97.58

3.4.2 PCA analysis

With the use of *PCA dimensionality reduction*, the most immediately noticeable thing is that, in the reduced domain, there is a whole island of samples in the train set that are outside the test domain.

As can be seen from Figure 3.15, the dense areas of faulty and healthy samples are quite homogeneous, and there are no substantial differences in values between those predicted for the training set and those for validation.

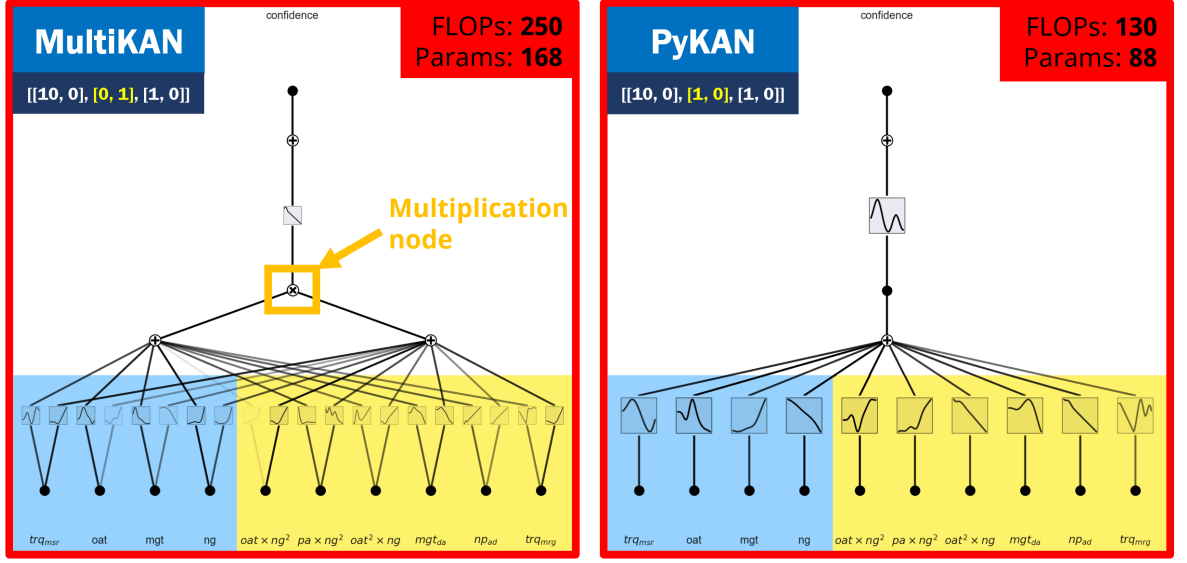


Figure 3.11: A KAN with a multiplication node, here referred to as "MultiKAN" and one without, referred to as "PyKAN" have been trained for fault detection. Whereas the former achieves a greater test score of 0.89, the latter is smaller and then more interpretable, but with a lower test score of 0.82.

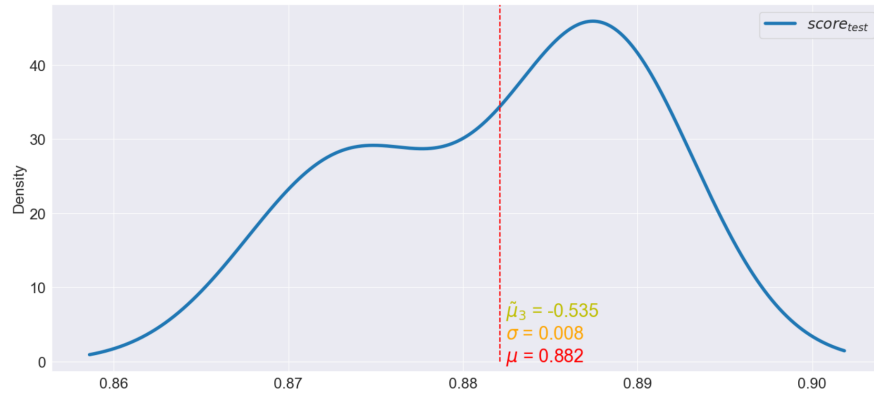


Figure 3.12: The 10 trainings show a low variance for the test loss.

3.4.3 t-SNE analysis

In order to carry out further analysis on the performance of the model, the *t-SNE* was also tested. In fact, *t-SNE* is much better than *PCA* at preserving the local structure of the data. However, this test was abandoned due to the complexity³ of doing it with our equipment.

3.4.4 KNN in PCA domain

The KNN employs the Euclidian distance between test points and labelled train points to make a prediction. The question driving this analysis is whether the test samples predicted as faulty by the PyKAN are near to faulty train samples.

³*t-SNE* has a time complexity of $\mathcal{O}(N^2)$, in his base form. However, the most popular implementation, that of *scikit-learn*, uses the Barnes-Hut approximation and achieves a time complexity of $\mathcal{O}(N \times \log(N))$. According to its documentation [Poličar, 2023], *Open TSNE* further improves the implementation by reducing it to linear complexity using the Flt-SNE approximation and by providing multi-core support. Unfortunately, even using *Open TSNE* was not enough to make the computation feasible.

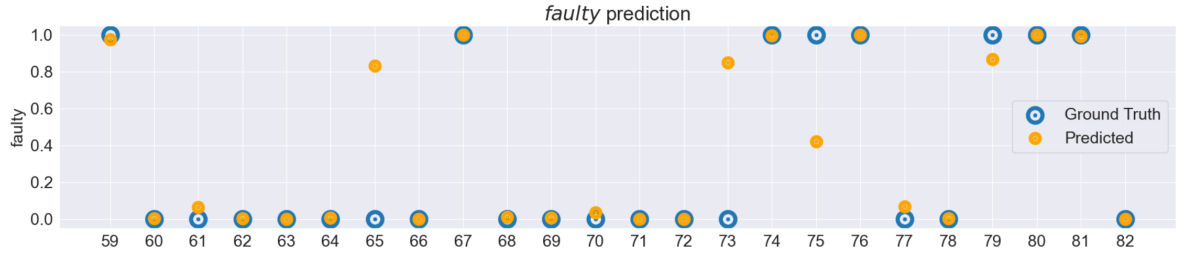


Figure 3.13: The prediction of the faulty state is a bit more disappointing than that of the $torque_{target}$.

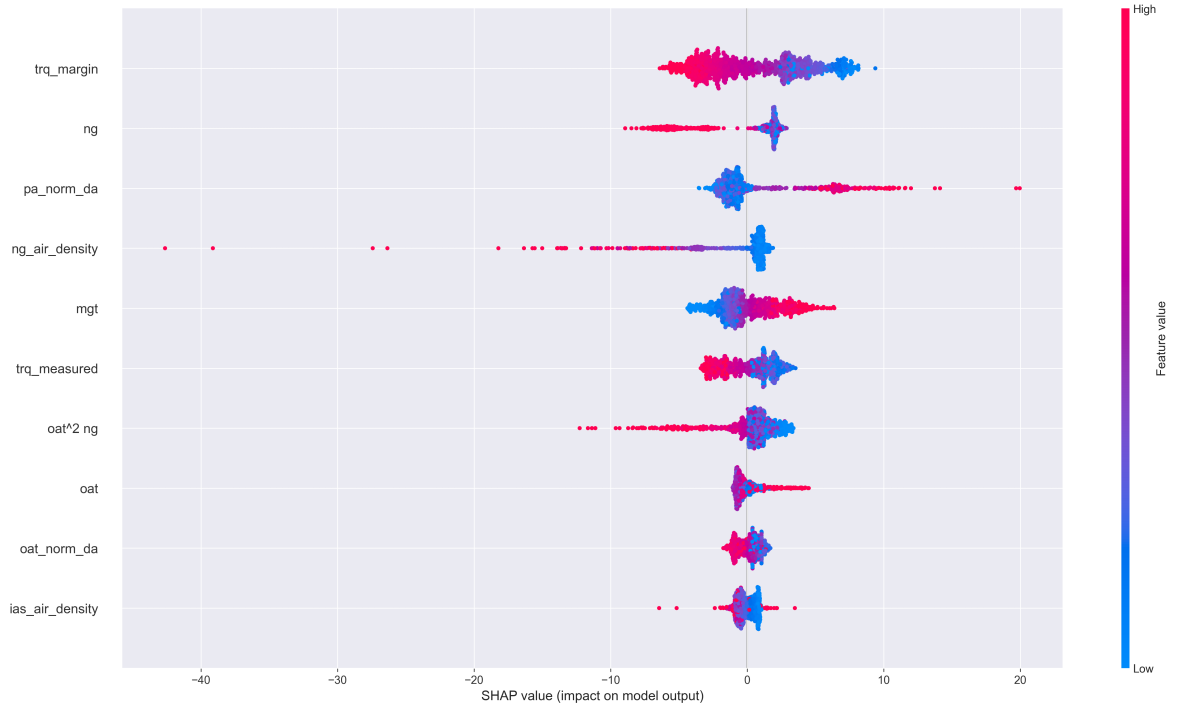


Figure 3.14: The Shapley value, computed for 2.000 random samples, shows how the KAN weights each feature in its predictions.

In order to do that, we compare the performance of our model of *MultiKAN* and a *300-NN*. It was found that the alignment score⁴ between them is 0.624.

Looking at the distributions shown in Figure 3.16, it can be seen that PyKAN has a more concentrated distribution at the ends, 0 and 1, while 300-NN has a more uniform distribution. This shows that the predictions of the first one are more reliable and the healthy and faulty classes are better separated, while 300-NN tends to remain more cautious.

So, we can say that PyKAN is generally *more confident* than the geometric classifier.

Also, we can see that **the KAN lost some confidence when queried on the test set**. This is no surprise, considering that the features domain of the test set significantly differs from that of the train set.

⁴Defined as

$$score := 1 - MAE = 1 - E[|\hat{y}_{KAN} - \hat{y}_{KNN}|]$$

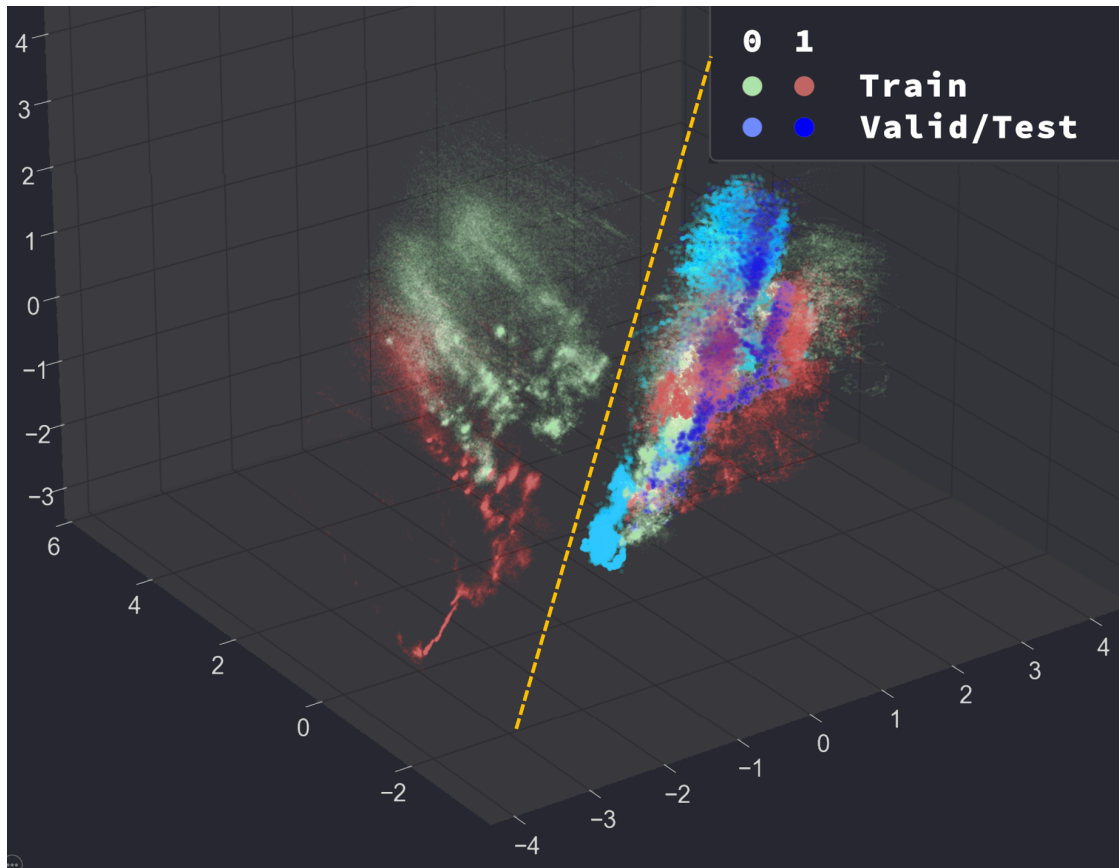


Figure 3.15: A 3D PCA plot shows the regions of the faulty samples of the test set alongside those of the train set.

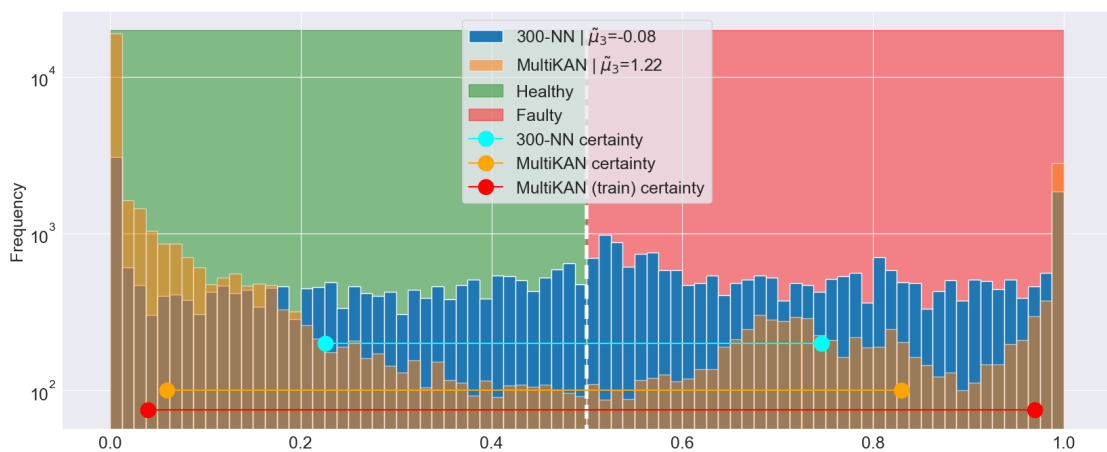


Figure 3.16: The distribution of the KAN's predictions over the test set are plotted next to those of the geometric classifiers. This gives an idea of how much test samples predicted to be faulty dwell near the faulty samples of the train set.

The Kolmogorov-Arnold Theorem in Machine Learning

Here we provide an introduction to the Kolmogorov-Arnold networks that we used as a model to solve the probabilistic regression and classification tasks in this challenge. If interested, the reader can read more at Liu [2024] and Cacciato et al. [2024].

A.1 The Kolmogorov-Arnold representation theorem

In 1957, Kolmogorov and Arnold disproved Hilbert's conjecture [Wikipedia, 2024a], which stated that 7th-degree equations cannot be solved using only functions of two variables, with their Kolmogorov-Arnold Theorem, or KAT for short.

The KAT states that every multivariate continuous function $f : [0, 1]^n \rightarrow \mathbb{R}$, with $n \geq 2$, can be written as a superposition of the two-argument addition of continuous functions of one variable, more specifically:

$$f(\vec{x}) = \sum_{q=0}^{2n} \chi_q \left(\sum_{p=1}^n \psi_{p,q}(x_p) \right)$$

where $\psi_{p,q} : [0, 1] \rightarrow \mathbb{R}$ and $\chi_q : \mathbb{R} \rightarrow \mathbb{R}$.

In its basic form, the KAT has much in common with the Universal Approximation Theorem, or UAT for short, which can be written in a very similar form:

$$f(\vec{x}) = \sum_{q=0}^{\infty} \omega_q \cdot \sigma \left(\sum_{p=1}^n \omega_{p,q} \cdot x_p \right)$$

First and foremost, they both give a representation of a multivariate function. However, whereas the UAT is an approximation, the KAT provides an exact representation of such a function, since both summations are limited. In a sense, it can be said that the power of the KAT over the UAT lies in this [Dhiman, 2024].

The UAT hides the non-linearity of the multivariate function in the activation function σ and calculates its argument as a linear combination of the inputs. The KAT, on the other hand, spreads the non-linearity over all the *inner* and *outer* functions, which can be non-smooth and even fractal. This "wild behaviour" makes these functions difficult to learn and has severely limited their use in machine learning [Wikipedia, 2024b].

A.2 The Kolmogorov-Arnold Networks

On 30 April 2024, Liu et al. published their work on Kolmogorov-Arnold networks, or KAN for short. KANs completely reinvent the MLP — a misnomer for multiple perceptrons since the latter implies a step activation function — architecture that has been widely used for over 60 years.

More specifically, inspired by KAT, **KANs replace linear weights on the edges with univariate functions parameterised as splines**, as can be seen in figure A.1.

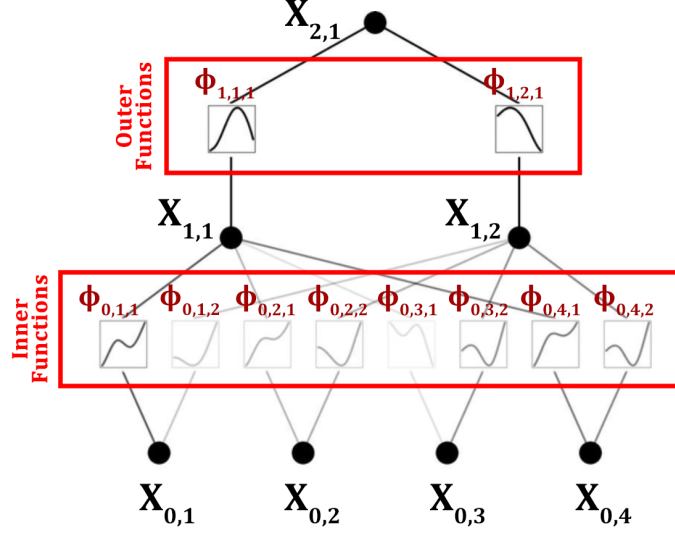


Figure A.1: A simple two-layer KAN, where the activation functions are arranged on the edges and the nodes compute the sum. The number of outer functions does not comply with the KAT statement, since they are less than $2n + 1 = 9$.

While MLPs have fixed activation functions on nodes (“neurons”), KANs have learnable activation functions on edges (“weights”) [Liu, 2024].

It should be noted, however, that, unlike the paper states, MLPs also have learnable activation functions, such as *parametric ReLU*, *Swish* and the more recent *Maxout*. Furthermore, if one defines the activation function of a layer as $\phi(x) = \sigma(Wx)$, then they have a learnable activation function in an MLP [Dhiman, 2024].

The architecture of KANs makes them significantly more computationally expensive than MLPs, but fortunately, they generally require far fewer parameters to achieve the same performance.

A.2.1 The paper’s real contribution

KANs are not the first neural networks to be inspired by the KAT, and splines have already been used in the past as learnable activation functions [Bohra *et al.*, 2020].

The real contribution of the paper lies in the extension of KAT — which in its basic form translates to a two-layer network — to an arbitrary number of layers with arbitrary widths. Furthermore, the choice of *B-form splines*, C^2 continuous by definition, as activation functions, makes all the operations necessary to compute the layer output differentiable, allowing training by back-propagation.

Unfortunately, restricting the KAT functions χ_q and $\psi_{p,q}$ with splines and relaxing the limit on the layer’s width violate the theorem statement and prevent obtaining an exact representation of the multivariate function.

However, **by stacking up multiple layers, KANs manage to reduce the well-known “wild behaviour” of the KAT functions**, making the choice of splines well justified [de Boor, 1978].

Furthermore, by decomposing the multivariate function into the sum of several univariate functions, KAN models increase their robustness to the curse of dimensionality.

A.2.2 The high interpretability of KAN

One of the aspects most emphasised in the paper is the interpretability of KANs. MLPs are notoriously black boxes, which means that while we can train and test them, it is not easy to understand what they have learned. So there is often a problem of trust when it comes to sensitive issues, like medical diagnoses.

Conversely, KANs require smaller architectures than MLPs to achieve the same performance, and **fewer parameters imply greater interpretability**. Furthermore, after training, they can be easily *pruned*, as shown in figure A.2, to remove the activation functions below a certain significance threshold in exchange for a small error. To make the pruning even more effective, a sparsification penalty is added to the loss function. This penalty is computed as a *L1 regularization*, which is defined for an activation function ψ as its average magnitude over its inputs.

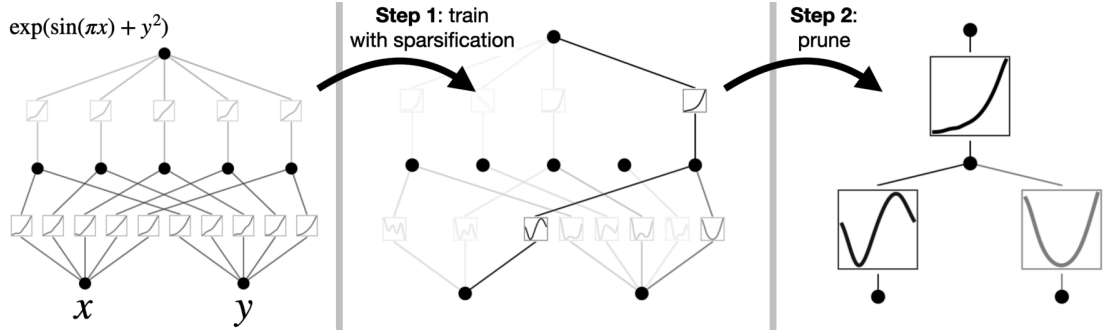


Figure A.2: After training with sparsification penalty, the nodes with *incoming* and *outgoing* scores — defined as the minimum and maximum *L1* norm for their activation functions — below a certain threshold are removed.

A.2.3 The introduction of MultKANs as a generalization of KANs

On 19 August 2024, three and a half months after the introduction of KAN, the same authors published a paper on a generalization of KAN called MultKAN [Liu *et al.*, 2024]. MultKAN improves the expressiveness of KAN to strengthen the link between KAN and science.

The main new feature introduced by MultKAN is the multiplicative node. The multiplicative nodes can be placed between the output nodes of a KAN layer, called *subnodes*, and the input nodes of the next layer, called *nodes*. It's up to the user to decide whether to use them or not, and how many to employ. Multiplicative nodes represent a structural change to the network architecture and do not introduce any learnable parameters. This means that the regularization techniques defined in the original KAN implementation are not impeded.

In code terms, the only difference with the original PyKAN implementation is the definition of the multiplication layer as

$$M_l(\mathbf{z}_l) = \underbrace{[\mathbf{z}_l^{(0)}, \mathbf{z}_l^{(1)}, \dots, \mathbf{z}_l^{(n_{l+1}^a)}]}_{\text{additive nodes}} \odot \underbrace{[\mathbf{z}_l^{(n_{l+1}^a+1)}, \dots, \mathbf{z}_l^{(n_{l+1}^a+n_{l+1}^m-1)}]}_{\text{multiplication nodes}} \odot \mathbf{z}_l^{(n_{l+1}^a+n_{l+1}^m)},$$

where $\odot$ is the element-wise multiplication, n_l^a and n_l^m are the numbers of addition and multiplication nodes of the l -th layer respectively.

The reader, can read more in Liu [2024] and Cacciatore *et al.* [2024].

Bibliography

- BLEALTAN (2024), «An Efficient Implementation of Kolmogorov-Arnold Network», URL <https://github.com/Blealtan/efficient-kan>. (Cited at page 9)
- BOHRA, P., CAMPOS, J., GUPTA, H., AZIZNEJAD, S. and UNSER, M. (2020), «Learning activation functions in deep (spline) neural networks», *IEEE Open Journal of Signal Processing*, vol. 1, p. 295–309. (Cited at page 17)
- CACCIATORE, A., MORELLI, V., PAGANICA, F., FRONTONI, E., MIGLIORELLI, L. and BERARDINI, D. (2024), «A preliminary study on continual learning in computer vision using Kolmogorov-Arnold Networks», URL <https://arxiv.org/abs/2409.13550>. (Cited at pages 16 e 18)
- DE BOOR, C. (1978), *A Practical Guide to Splines*, URL <https://link.springer.com/book/9780387953663>. (Cited at page 17)
- DHIMAN, V. (2024), «KAN: Kolmogorov–Arnold Networks: A review», URL https://vikasdhiman.info/reviews/KAN_a_review.pdf. (Cited at pages 16 e 17)
- LIU, Z. (2024), «KAN: Kolmogorov-Arnold Networks», URL <https://arxiv.org/abs/2404.19756>. (Cited at pages 9, 16, 17 e 18)
- LIU, Z., MA, P., WANG, Y., MATUSIK, W. and TEGMARK, M. (2024), «KAN 2.0: Kolmogorov-Arnold Networks Meet Science», URL <https://arxiv.org/abs/2408.10205>. (Cited at page 18)
- OZEKI, M. T. S. T. W. K., K. and NAKAMURA, T. (2024), «Estimating the health of turbine engine based on the relationship between torque margin and density altitude.», URL <https://papers.phmsociety.org/index.php/phmconf/article/view/4191>. (Cited at page 1)
- PHMSOCIETY (2024), «PHM North America 2024 Conference Data Challenge», URL <https://data.phmsociety.org/phm2024-conference-data-challenge>. (Cited at page 10)
- POLIČAR, P. (2023), «Comparison to other t-SNE implementations», URL <https://opentsne.readthedocs.io/en/stable/benchmarks.html>. (Cited at page 13)
- ROMANO, S. N. W. I. S. H. W. . S. R., T. (2024), «Estimating The Health of Helicopter Turbine Engines by Means of Regression and Classification Using a Probabilistic Neural Network», URL <https://papers.phmsociety.org/index.php/phmconf/article/view/4196>. (Cited at page 9)

WIKIPEDIA (2024a), «Hilbert's thirteenth problem», URL https://en.wikipedia.org/w/index.php?title=Hilbert's_thirteenth_problem. (Cited at page 16)

WIKIPEDIA (2024b), «Kolmogorov–Arnold representation theorem», URL https://en.wikipedia.org/w/index.php?title=Kolmogorov-Arnold_representation_theorem. (Cited at page 16)